

DOCUMENTACIÓN SEGURA

Funciones del programa de cifrado César y Atbash

Un solo documento con los 23 identificadores, contratos, validaciones, efectos, seguridad y complejidad del código.

Este documento evita capturas y copias completas del código. Cada ID aparece inmediatamente encima de su función en los archivos JavaScript y se explica aquí sin incluir secretos ni datos personales.

Versión consolidada - 23 funciones documentadas

Índice

Índice	2
Criterios de documentación segura	3
Configuración del alfabeto	4
CFG-GRAPHEME-SEGMENT-001 — segmentarGrafemas	5
CFG-ALPHABET-001 — crearAlfabeto	6
Cifrados	7
CIPHER-SHIFT-NORMALIZE-001 — normalizarDesplazamiento	8
CIPHER-PASSTHROUGH-001 — transformarSoloActivos	9
CIPHER-CAESAR-ENC-001 — cifrarCesar	10
CIPHER-CAESAR-DEC-001 — descifrarCesar	11
CIPHER-ATBASH-001 — transformarAtbash	12
Análisis automático	13
ANALYSIS-TEXT-NORMALIZE-001 — textoAnalizable	14
ANALYSIS-LANGUAGE-SCORE-001 — puntuarEspañol	15
ANALYSIS-CANDIDATES-001 — generarCandidatos	16
ANALYSIS-AUTO-DETECT-001 — detectarYDescifrar	17
Interfaz y validación	18
UI-QUERY-001 — \$	19
UI-SYMBOL-NAME-001 — nombreVisible	20
UI-MESSAGE-001 — mostrarMensaje	21
UI-CHARSET-SELECTOR-001 — reconstruirSelector	22
UI-ACTIVE-ALPHABET-001 — alfabetoActivo	23
VAL-TEXT-001 — validarTexto	24
UI-SAFE-OUTPUT-001 — mostrarResultadoSeguro	25
UI-METHOD-STATE-001 — actualizarEstadoMetodo	26
UI-ENCRYPT-001 — manejarCifrado	27
UI-DECRYPT-001 — manejarDescifrado	28
UI-COPY-001 — copiarSalida	29
UI-INIT-001 — iniciar	30

Criterios de documentación segura

- Cada función real tiene un ID único y visible en el código.
- Cada ID se documenta una sola vez dentro de este PDF.
- No se incluyen contraseñas, tokens, claves privadas, rutas personales ni datos sensibles.
- Las explicaciones describen el contrato y el comportamiento sin duplicar archivos completos.
- Las entradas del usuario se tratan como texto y la interfaz utiliza `textContent`.
- La documentación corresponde a 23 funciones distribuidas en cuatro módulos JavaScript.

Configuración del alfabeto

Esta sección reúne 2 funciones relacionadas con configuración del alfabeto.

CFG-GRAPHEME-SEGMENT-001 —

segmentarGrafemas

ID: CFG-GRAPHEME-SEGMENT-001

Campo	Valor
Archivo	programa_web/js/alphabet.js
Tipo	Función pura exportada
Responsabilidad	Separar texto en unidades visibles

Qué hace

Recibe una cadena y devuelve un arreglo de grafemas. Utiliza `Intl.Segmenter` con granularidad `grapheme` cuando el navegador lo permite; como alternativa utiliza `Array.from` para conservar puntos de código sin reescribirlos.

Contrato

- Entrada: una cadena de texto.
- Salida: arreglo ordenado de cadenas, una por unidad segmentada.
- Error: `TypeError` cuando el argumento no es una cadena.

Importancia

Evita recorrer texto únicamente por unidades UTF-16. Esto mejora el tratamiento de tildes combinadas, emojis y otros símbolos Unicode. La función no normaliza ni corrige el contenido, por lo que respeta la coincidencia literal exigida por el charset.

Seguridad y complejidad

No usa HTML, red ni almacenamiento. Tiempo y memoria $O(n)$ respecto al tamaño del texto.

CFG-ALPHABET-001 — crearAlfabeto

ID: CFG-ALPHABET-001

Campo	Valor
Archivo	programa_web/js/alphabet.js
Tipo	Función pura exportada
Responsabilidad	Validar y construir el charset activo

Qué hace

Convierte una cadena o arreglo de símbolos en un objeto de alfabeto con tres propiedades: lista ordenada `simbolos`, mapa `indice` y cantidad `modulo`. El orden recibido define las posiciones usadas por César y Atbash.

Contrato

- Entrada: cadena o arreglo de grafemas activos.
- Salida: objeto { `simbolos`, `indice`, `modulo` }.
- Errores: menos de dos símbolos, más de 4096, elementos vacíos o repetidos.

Seguridad

La coincidencia es literal: no ejecuta contenido ni normaliza Unicode. El arreglo y el objeto exterior se congelan para evitar modificaciones accidentales. El límite de 4096 evita crecimiento ilimitado.

Complejidad

Tiempo y memoria $O(m)$, donde m es el número de símbolos activos.

Cifrados

Esta sección reúne 5 funciones relacionadas con cifrados.

CIPHER-SHIFT-NORMALIZE-001 —

normalizarDesplazamiento

ID: CIPHER-SHIFT-NORMALIZE-001

Campo	Valor
Archivo	programa_web/js/ciphers.js
Tipo	Función pura exportada

Qué hace

Convierte cualquier desplazamiento entero César en un valor válido entre cero y `modulo - 1`. Aplica $((k \% modulo) + modulo) \% modulo$, por lo que también admite valores negativos o mayores que el alfabeto.

Contrato

- Entradas: entero `k` y entero `modulo` mayor o igual que dos.
- Salida: entero normalizado.
- Errores: `TypeError` para un desplazamiento no entero y `RangeError` para un módulo inválido.

Seguridad y complejidad

No modifica estado ni procesa contenido del usuario. Tiempo y memoria constantes, $O(1)$.

CIPHER-PASSTHROUGH-001 —

transformarSoloActivos

ID: CIPHER-PASSTHROUGH-001

Campo	Valor
Archivo	programa_web/js/ciphers.js
Tipo	Función pura exportada

Qué hace

Es la función central de transformación. Recorre la entrada por grafemas, consulta el mapa del alfabeto y transforma únicamente los símbolos activos. Un símbolo ausente se devuelve exactamente igual y conserva su posición: esta es la regla **passthrough**.

Contrato

- Entradas: texto, alfabeto válido y función `resolverIndice`.
- Salida: nueva cadena transformada.
- Errores: texto, alfabeto o función inválidos; índice calculado fuera del charset.

Seguridad

No interpreta HTML ni ejecuta símbolos. Valida que cada índice de destino sea entero y esté dentro del módulo. Los caracteres externos, incluidos saltos, tildes y emojis, no se alteran.

Complejidad

Tiempo y memoria $O(n)$ respecto a los grafemas del mensaje.

CIPHER-CAESAR-ENC-001 — cifrarCesar

ID: CIPHER-CAESAR-ENC-001

Campo	Valor
Archivo	programa_web/js/ciphers.js
Tipo	Función pura exportada

Qué hace

Cifra un texto desplazando cada símbolo activo k posiciones hacia delante. Normaliza el desplazamiento y delega el recorrido a `transformarSoloActivos`.

Regla

$\text{destino} = (\text{posición} + \text{desplazamiento}) \bmod \text{módulo}.$

Contrato

- Entradas: texto, entero k y alfabeto validado.
- Salida: criptograma César.
- Passthrough: todo símbolo inactivo permanece intacto.

Seguridad y complejidad

No ofrece seguridad criptográfica moderna; se usa con fines educativos. Tiempo y memoria $O(n)$.

CIPHER-CAESAR-DEC-001 — descifrarCesar

ID: CIPHER-CAESAR-DEC-001

Campo	Valor
Archivo	programa_web/js/ciphers.js
Tipo	Función pura exportada

Qué hace

Revierte César restando el desplazamiento a la posición de cada símbolo activo. Usa la misma normalización y regla passthrough que el cifrado.

Regla

$\text{destino} = (\text{posición} - \text{desplazamiento} + \text{módulo}) \bmod \text{módulo}.$

Contrato

- Entradas: criptograma, entero `k` y alfabeto validado.
- Salida: texto descifrado.
- Propiedad: con el mismo alfabeto y `k`, revierte el resultado de `cifrarCesar`.

Complejidad

Tiempo y memoria $O(n)$.

CIPHER-ATBASH-001 — `transformarAtbash`

ID: CIPHER-ATBASH-001

Campo	Valor
Archivo	<code>programa_web/js/ciphers.js</code>
Tipo	Función pura exportada

Qué hace

Sustituye cada símbolo activo por el ubicado en la posición opuesta del alfabeto. La regla es $\text{módulo} - 1 - \text{posición}$.

Contrato

- Entradas: texto y alfabeto validado.
- Salida: texto transformado.
- Propiedad involutiva: aplicar la función dos veces recupera el original.
- Passthrough: símbolos inactivos no cambian.

Seguridad y complejidad

Atbash no tiene una clave secreta y no protege información real. Tiempo y memoria $O(n)$.

Análisis automático

Esta sección reúne 4 funciones relacionadas con análisis automático.

ANALYSIS-TEXT-NORMALIZE-001 —

textoAnalizable

ID: ANALYSIS-TEXT-NORMALIZE-001

Campo	Valor
Archivo	programa_web/js/analysis.js
Tipo	Función privada y pura

Qué hace

Prepara una copia de un candidato para analizarlo: convierte a minúsculas con configuración española, aplica normalización NFD, elimina marcas combinantes y reemplaza caracteres no lingüísticos por espacios.

Contrato

- Entrada: texto candidato.
- Salida: copia simplificada para el modelo de español.
- Efecto sobre el resultado: ninguno; el texto original nunca se modifica ni se muestra normalizado.

Seguridad

La separación entre copia analítica y salida evita reescribir tildes o símbolos del mensaje recuperado. No usa red, HTML ni evaluación dinámica.

Complejidad

Tiempo y memoria $O(n)$.

ANALYSIS-LANGUAGE-SCORE-001 —

puntuarEspañol

ID: ANALYSIS-LANGUAGE-SCORE-001

Campo	Valor
Archivo	programa_web/js/analysis.js
Tipo	Función pura exportada

Qué hace

Asigna a un candidato una puntuación comparable según su parecido con el español. Combina chi-cuadrada de frecuencias, palabras comunes, n-gramas, secuencias raras, proporción de vocales, cobertura de letras y penalización de símbolos no lingüísticos.

Contrato

- Entrada: texto candidato.
- Salida: número; un valor mayor representa mejor ajuste.
- Caso sin letras: devuelve `-Infinity`.

Seguridad y limitaciones

El análisis es local y determinista. Es una heurística, no una prueba matemática ni un porcentaje de certeza. La penalización de símbolos evita falsos positivos en charsets Unicode amplios.

Complejidad

Tiempo y memoria $O(n)$ por candidato.

ANALYSIS-CANDIDATES-001 —

generarCandidatos

ID: ANALYSIS-CANDIDATES-001

Campo	Valor
Archivo	programa_web/js/analysis.js
Tipo	Función pura exportada

Qué hace

Genera todas las hipótesis permitidas: una transformación Atbash y un descifrado César para cada desplazamiento desde 1 hasta `módulo - 1`.

Contrato

- Entradas: criptograma y alfabeto validado.
- Salida: arreglo de objetos `{ metodo, desplazamiento, texto }`.
- Cantidad: exactamente `módulo` candidatos.

Seguridad y rendimiento

No presenta candidatos en el DOM ni ejecuta contenido. Con charsets grandes el costo crece; la validación limita el alfabeto a 4096 grafemas.

Complejidad

Tiempo y memoria aproximados $O(m \cdot n)$, con `m` como módulo y `n` como longitud del texto.

ANALYSIS-AUTO-DETECT-001 —

detectarYDescifrar

ID: ANALYSIS-AUTO-DETECT-001

Campo	Valor
Archivo	programa_web/js/analysis.js
Tipo	Función orquestadora exportada

Qué hace

Genera hipótesis, puntúa cada texto, ordena las puntuaciones finitas de mayor a menor y devuelve únicamente la mejor. Las puntuaciones no finitas se envían al final. En empate real prefiere Atbash y después el menor desplazamiento.

Contrato

- Entradas: criptograma y alfabeto validado.
- Salida: { metodo, desplazamiento, texto, confianza }.
- Confianza: alta, media o baja según la diferencia entre los dos mejores puntajes.

Seguridad y limitaciones

Las alternativas nunca llegan a la interfaz. La confianza es una etiqueta heurística; mensajes cortos, otro idioma o un charset incorrecto pueden producir ambigüedad.

Complejidad

Dominada por la generación y evaluación: aproximadamente $O(m \cdot n)$.

Interfaz y validación

Esta sección reúne 12 funciones relacionadas con interfaz y validación.

UI-QUERY-001 — \$

ID: UI-QUERY-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función auxiliar de interfaz

Qué hace

Recibe un selector CSS y devuelve el primer elemento del documento que coincide mediante `document.querySelector`.

Contrato

- Entrada: selector CSS usado internamente por la aplicación.
- Salida: elemento DOM o `null`.
- Efectos: ninguno; solo consulta el documento.

Seguridad

Los selectores están definidos en el código y no provienen de texto libre del usuario. La función no inserta HTML ni evalúa JavaScript.

UI-SYMBOL-NAME-001 — nombreVisible

ID: UI-SYMBOL-NAME-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función pura de presentación

Qué hace

Convierte espacio, salto de línea y tabulador en etiquetas comprensibles para las casillas del charset. Para cualquier otro símbolo devuelve el valor original.

Contrato

- Entrada: un grafema.
- Salida: texto visible para la interfaz.

Seguridad

La etiqueta se asigna posteriormente con `textContent`, por lo que un símbolo no puede convertirse en HTML ejecutable. Complejidad $O(1)$.

UI-MESSAGE-001 — `mostrarMensaje`

ID: UI-MESSAGE-001

Campo	Valor
Archivo	<code>programa_web/js/app.js</code>
Tipo	Función con efecto en el DOM

Qué hace

Actualiza el aviso general de la aplicación y registra su tipo visual (`info`, `ok` o `error`) en un atributo de datos.

Contrato

- Entradas: texto del aviso y tipo opcional.
- Salida: ninguna.
- Efecto: modifica contenido y atributo del elemento `#aviso`.

Seguridad

Utiliza `textContent`; por lo tanto, los mensajes de error derivados de entradas no se interpretan como HTML.

UI-CHARSET-SELECTOR-001 —

reconstruirSelector

ID: UI-CHARSET-SELECTOR-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función de interfaz con estado

Qué hace

Lee el charset escrito, lo segmenta, elimina duplicados conservando la primera aparición y marca todos los símbolos resultantes como activos. Después crea una casilla por símbolo mediante nodos DOM y actualiza el contador.

Entradas y efectos

No recibe parámetros. Lee `#charset`, **actualiza** `estado.candidatos`, `estado.activos`, `#charset-grid` y `#contador-charset`.

Seguridad

Crea elementos con `createElement` y asigna etiquetas mediante `textContent`; nunca concatena HTML. Los duplicados se eliminan antes de construir el alfabeto.

Complejidad

Tiempo y memoria $O(m)$.

UI-ACTIVE-ALPHABET-001 — alfabetoActivo

ID: UI-ACTIVE-ALPHABET-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función de integración

Qué hace

Filtra los candidatos del charset según el conjunto de casillas activas y entrega el resultado a `crearAlfabeto`.

Contrato

- Entrada: estado interno de la interfaz.
- Salida: alfabeto validado { `simbolos`, `indice`, `modulo` }.
- Errores: propaga las validaciones de `crearAlfabeto`, por ejemplo menos de dos símbolos activos.

Seguridad

Mantiene el orden original y no acepta directamente índices suministrados por el usuario.

Complejidad $O(m)$.

VAL-TEXT-001 — validarTexto

ID: VAL-TEXT-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función de validación

Qué hace

Comprueba que un mensaje no supere el límite configurado de 20,000 unidades de cadena antes de cifrar o generar candidatos de descifrado.

Contrato

- Entrada: texto del formulario.
- Salida: ninguna cuando es válido.
- Error: `RangeError` cuando excede el límite.

Seguridad

Reduce el riesgo de bloquear la interfaz con entradas excesivas. La validación se realiza antes del trabajo criptográfico costoso. Complejidad $O(1)$ para consultar la longitud.

UI-SAFE-OUTPUT-001 —

mostrarResultadoSeguro

ID: UI-SAFE-OUTPUT-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función con efecto en el DOM

Qué hace

Escribe el resultado y su detalle en los elementos indicados. Se usa tanto para presentar respuestas como para limpiar salidas después de un error.

Contrato

- Entradas: IDs de salida y detalle, texto y descripción opcionales.
- Salida: ninguna.
- Efecto: modifica dos elementos del DOM.

Seguridad

Usa exclusivamente `textContent`. Una entrada que contenga etiquetas o scripts se muestra como texto literal y no se ejecuta.

UI-METHOD-STATE-001 —

actualizarEstadoMetodo

ID: UI-METHOD-STATE-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función de presentación

Qué hace

Consulta el método seleccionado. Si es César, muestra y habilita el desplazamiento; para Atbash lo oculta y deshabilita porque ese algoritmo no usa clave numérica.

Contrato

No recibe parámetros ni devuelve valor. Modifica las propiedades `hidden` y `disabled` de los controles relacionados.

Seguridad

Solo cambia el estado visual; la función de cifrado vuelve a validar el método antes de procesar el mensaje.

UI-ENCRYPT-001 — manejarCifrado

ID: UI-ENCRYPT-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Controlador de evento

Qué hace

Impide el envío tradicional del formulario, valida el mensaje, construye el alfabeto activo y ejecuta César o Atbash. Después presenta el resultado, módulo y desplazamiento cuando corresponde.

Contrato

- Entrada: evento `submit`.
- Salida: ninguna; actualiza la interfaz.
- Errores controlados: método inválido, alfabeto inválido, texto excesivo o desplazamiento equivalente a cero.

Seguridad

Procesa localmente, limita entradas y delega la salida a `mostrarResultadoSeguro`. Los caracteres inactivos permanecen intactos.

UI-DECRYPT-001 — manejarDescifrado

ID: UI-DECRYPT-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Controlador de evento

Qué hace

Valida el criptograma, construye el alfabeto activo y llama a `detectarYDescifrar`. Muestra exclusivamente la mejor hipótesis junto con método, desplazamiento César y confianza.

Contrato

- Entrada: evento `submit`.
- Salida: ninguna; actualiza la interfaz.
- Errores: se capturan, se limpia la salida y se presenta un aviso seguro.

Seguridad y rúbrica

No expone la lista de candidatos ni solicita que el usuario elija uno. Todo el análisis ocurre localmente y la salida usa `textContent`.

UI-COPY-001 — copiarSalida

ID: UI-COPY-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función asíncrona de interfaz

Qué hace

Lee el texto visible de una salida y solicita al navegador copiarlo mediante `navigator.clipboard.writeText`. Informa éxito, ausencia de contenido o denegación del permiso.

Contrato

- Entrada: ID del elemento de salida.
- Salida: promesa resuelta sin valor útil.
- Efecto externo: escritura en el portapapeles autorizada por el navegador.

Seguridad

Solo copia `textContent`; no lee archivos ni envía información por red. Los fallos de permiso se capturan sin mostrar detalles internos.

UI-INIT-001 — iniciar

ID: UI-INIT-001

Campo	Valor
Archivo	programa_web/js/app.js
Tipo	Función de inicialización

Qué hace

Carga el preset inicial, construye el selector, sincroniza el método y registra todos los eventos de la página: cambio de preset, edición del charset, activación de casillas, selección de método, formularios y botones de copia.

Contrato

No recibe parámetros ni devuelve valor. Requiere que el DOM definido en `index.html` esté disponible cuando se ejecuta.

Seguridad y efectos

Centraliza los puntos de entrada de la interfaz. Los eventos delegan en funciones documentadas que validan entradas y usan salidas seguras. Se ejecuta una sola vez al cargar el módulo.